

第八章 NP完全问题



8.1 什么是好算法？

算法的种类和数量积累到一定程度时，需要对算法进行比较和分类。

什么是好算法？Edmonds, 1975年提出了一个被沿用至今的标准。**Edmonds算法标准**指出具有多项式时间的算法为好算法。

多项式时间算法：如果 Π 是任意一个问题，对 Π 存在着一个算法，它的时间复杂性为 $O(n^k)$ ，其中 n 为输入规模， k 为非负整数，就认为存在着一个解问题 Π 的多项式时间算法。

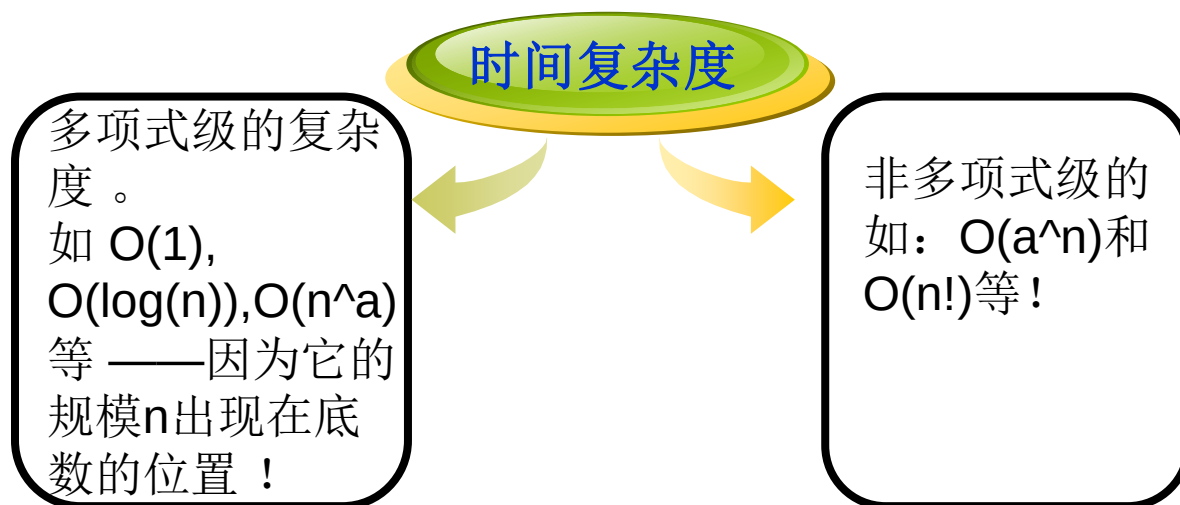
以多项式作为分界函数？

原因有两个：

一、常见算法大致分为两类：

- 一类是多项式时间内可实现的
- 另一类需要指数时间 ($O(c^n)$)

多项式时间算法的可实现性远大于指数时间算法。



8.2 P类问题—易解的问题

是否每个问题都有多项式时间算法？

在考虑问题的计算复杂性时，常把它化为相应的判定问题考虑。

首先看问题分类及其转换。

问题分类

□ 一类是判定问题

解只有两种，yes或no。

例：给定图 $G=(V, E)$ ，问该图是否有哈密尔顿圈。

□ 一类是优化问题

例：给定图 $G=(V, E)$ ，假设边的费用为自然数。求该图的最短哈密尔顿回路。

问题转换

优化问题可转换为相应的判定问题求解。

例：给定图 $G=(V, E)$ ，假设边的费用为自然数。给定 $k=1, 2, \dots$ ，问是否有长度不超过 k 的哈密尔顿回路。

P类问题

P类:

具有多项式时间算法的判定问题形成一个计算复杂类，记为P类。

P类—易解的问题

P-Polynomial

思考：已学知识中哪些问题属P类问题？

8.3 NP类问题—难解的问题

具有指数时间算法的问题。

例：货郎担问题(TSP问题)。

$n!$ 排列方式。

$n=6$: $6! = 720$

$n=19$: $19! \approx 1.21 \times 10^{17}$

每秒排一次，排 3.84×10^9 年

每秒排百万次，排3000年

有算法但实现性差。

TSP问题

1998年，解决了美国13509个城市之间的TSP问题

2001年，解决了德国15112个城市之间的TSP问题

解决**15112个城市之间的TSP问题**，共使用了美国**Rice大学**和**普林斯顿大学**之间网络互连的，由速度为**500MHz** 的**Compaq EV6 Alpha** 处理器组成的**110**台计算机，所有计算机花费的时间之和为**22.6年**。

NP类问题

一般而言，验证解比求解易。

对具有指数时间的问题，有些可用不确定性算法求解。该算法包含两个阶段：

- 推测阶段

对规模为 n 的输入实例 x ，产生一个输出 y 。

- 验证阶段

检验 y 是否满足解形式，是否是解。

NP类问题

推测阶段是具有多项式时间的非确定性 (non-determinism) 算法，对输入实例 x ，下次产生的输出可能不是 y 。

验证阶段是具有多项式时间的确定性算法。

NP类问题

NP类:

由具有多项式时间的非确定性算法求解的判定问题形成的一个计算复杂类，记为NP类。

NP—难解的问题

NP—Nondeterministic Polynomial

NP类问题举例—货郎担问题

例：货郎担的判定问题：给定 n 个城市、正常数 k 及城市之间的费用矩阵 C ，判定是否存在一条经过所有城市一次且仅一次，最后返回初始出发城市且费用小于常数 k 的回路。

- 算法A用非确定算法在多项式时间内推测一条回路
- A用确定算法在多项式时间内判定回路是否是哈密尔顿回路，是否费用和小于 k ，返回yes或no。

P类和NP类问题的差别

主要差别在于：

- P类问题可以用多项式时间的确定性算法进行判定或求解
- NP类问题可以用多项式时间的确定性算法来检查和验证它的解

结论： $P \subseteq NP$

一个可以多项式时间的用确定算法进行判定或求解问题，则一定可以多项式时间的用确定性算法来检查和验证它的解。

猜测： $NP \neq P$?

8.4 NPC问题

NPC—NP Complete, NP完全

为定义NP完全问题，首先给出归约的定义。

定义8.1 令 Π 和 Π' 是两个判定问题，如果存在一个具有如下性能的确定性算法A，可以用多项式的时间，把问题 Π' 的实例 I' 转化为问题 Π 的实例 I ，使得 I' 的答案为yes，当且仅当 I 的答案是yes，就称 Π' 以多项式时间归约于 Π ，记为 $\Pi' \in_p \Pi$ 。

一个问题 Π' 可以约化为问题 Π 的含义即是，可以用解决问题 Π 的解法来解决问题 Π' ，或者说，问题 Π' 可以“变成”问题 Π 。如：一元一次方程可以“归约”为一元二次方程。

关于归约

根据定义8.1，由于多项式与多项式的复合仍为多项式，因此可推得：

1) 两个判定问题 Π 、 Π'

■ 若 $\Pi \in P$ ，且 $\Pi' \propto_p \Pi$ ，则 $\Pi' \in P$

2) 归约关系 $\propto_p$ 满足传递性：三个判定问题 Π 、 Π' 、 Π'' ，若 $\Pi'' \propto_p \Pi'$ ， $\Pi' \propto_p \Pi$ ，则 $\Pi'' \propto_p \Pi$ 。

NP完全问题定义

定义8.2 令 Π 是一个判定问题，如果对NP中的每一个问题 $\Pi' \in \text{NP}$ ，有 $\Pi' \propto_p \Pi$ ，就称判定问题 Π 是一个NP难题。

定义8.3 令 Π 是一个判定问题，如果：

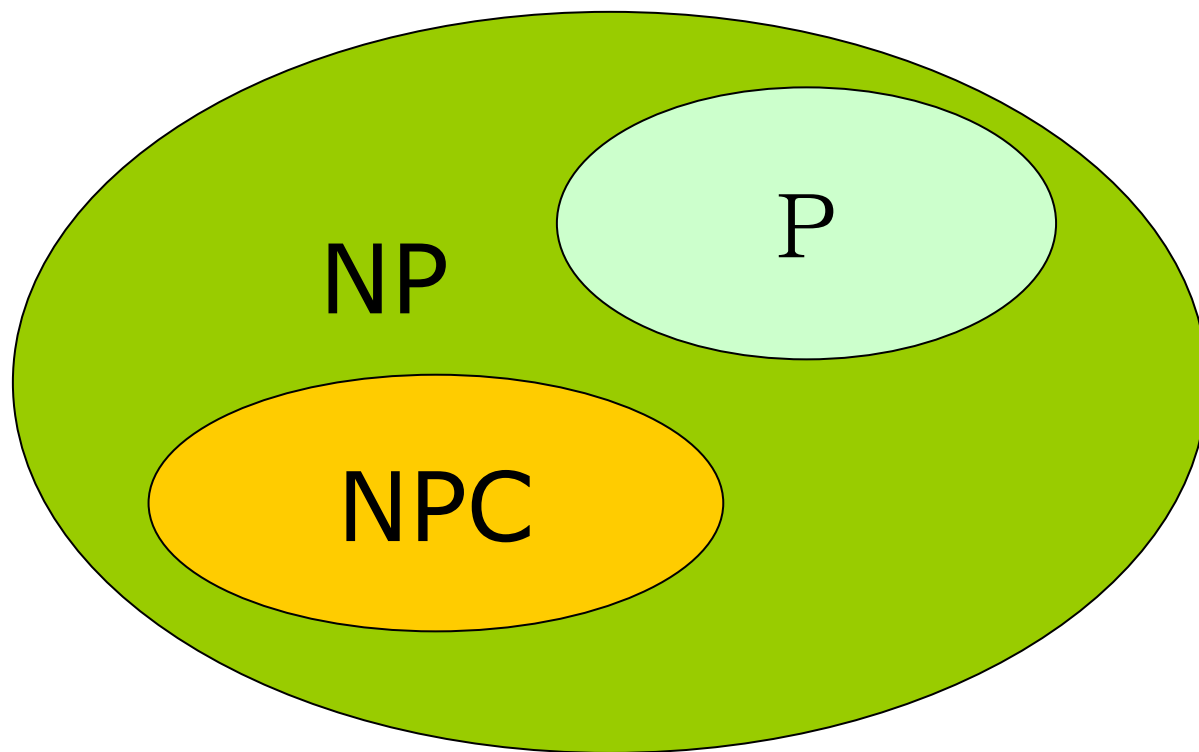
(1) $\Pi \in \text{NP}$;

(2) 对NP中的所有问题 $\Pi' \in \text{NP}$ ，都有
 $\Pi' \propto_p \Pi$;

则称判定问题 Π 是NP完全 (NPC)的。

P类、NP类、NPC类问题关系

根据定义，可用如下图表示三者之间的关系：



P类、NP类、NPC类问题关系

□ 对NPC问题，有个重要性质

对NPC类中的一个问题，如果能够证明用多项式时间的确定性算法来进行求解或判定，那么，NP中的所有问题都可以通过多项式时间的确定性算法来进行求解或判定。

NP完全问题的证明

定理8.1 令 Π 和 Π' 是NP中的两个问题，使得
 $\Pi' \propto_p \Pi$ 。如果 Π' 是NP完全的，则 Π 也是NP完全的。

根据定理8.1，为证明问题 Π 是NP完全的，只需证明下列两点：

(1) $\Pi \in \text{NP}$

(2) 存在一个NP完全问题 Π' ，使得 $\Pi' \propto_p \Pi$ 。

8.5 几个典型的NPC问题

□ 可满足性问题 (SATISFIABILITY)

判定问题: SATISFIABILITY

输入: 合取范式 (CNF) 布尔表达式 f

问题: 对布尔表达式 f 中的布尔变量赋值, 是否可使 f 的真值为真。

$$\text{例: } f = (x_2 \vee x_3 \vee x_5) \wedge (x_1 \vee \overline{x_3} \vee x_4 \vee x_5) \wedge (\overline{x_2} \vee \overline{x_3} \vee x_4)$$

Cook定理

定理8.2: 可满足性问题SATISFIABILITY是NP完全的。(Cook定理, 1971年)

Stephen Arthur Cook—1982年获图灵奖, NP完全性理论的奠基人

Cook定理的意义: 给出了第一个NP完全问题, 使得对任何问题 Π , 只要能证明 $\Pi \in \text{NP}$, 且 $\text{SAT} \propto_p \Pi$, 那么 Π 是NPC问题。为NPC问题的发现奠定了基础。

部分NP完全问题树

以**SAT**为基础，很快地又证明了很多其他的NP完全问题，逐渐形成了一颗以**SAT**为根的NP完全树。图8.1给出了这颗树的一小部分，其中每个结点都是一个NP完全问题，该问题可在多项式时间里，转换为它的任一儿子结点所表示的问题。

部分NP完全问题树

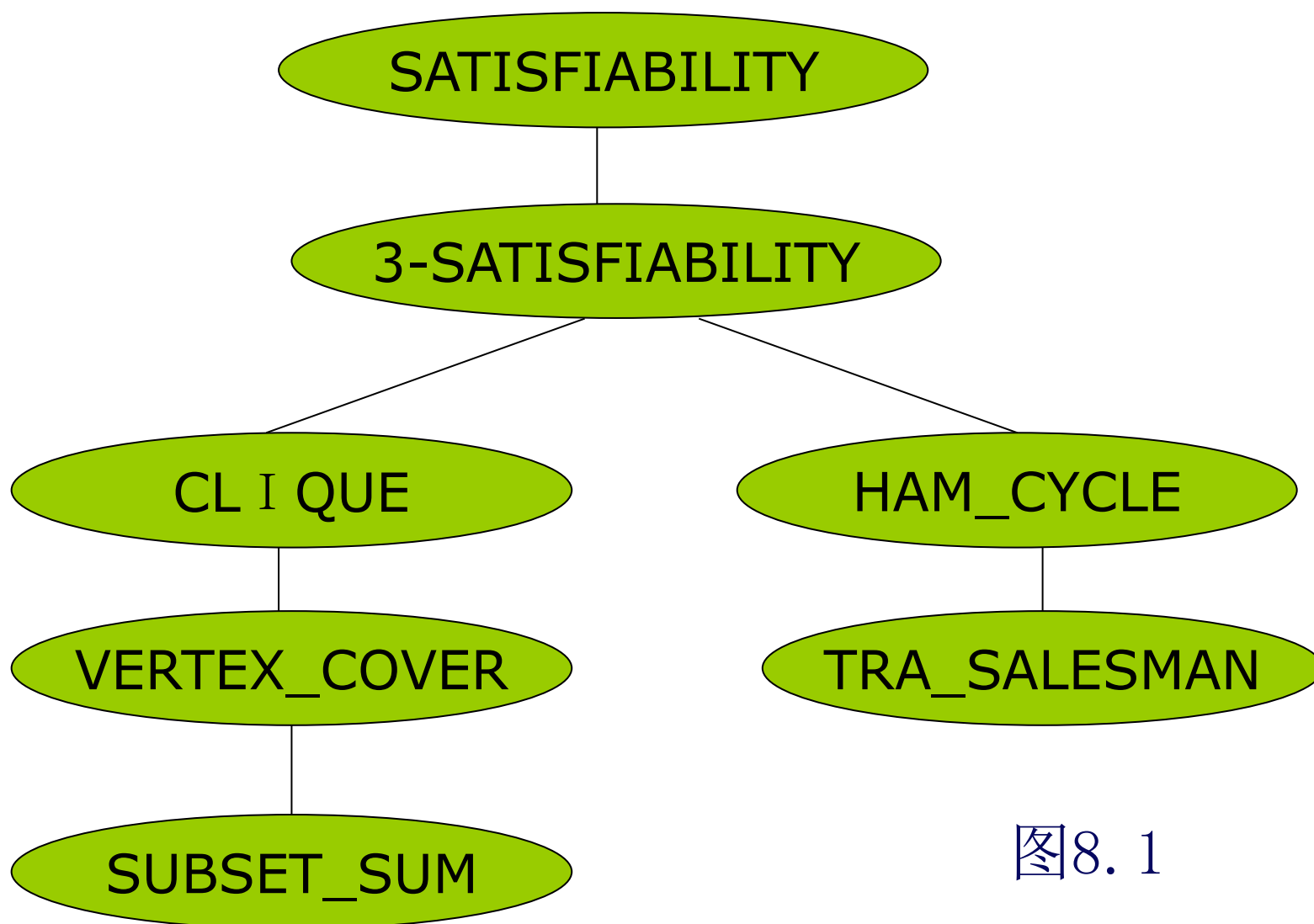
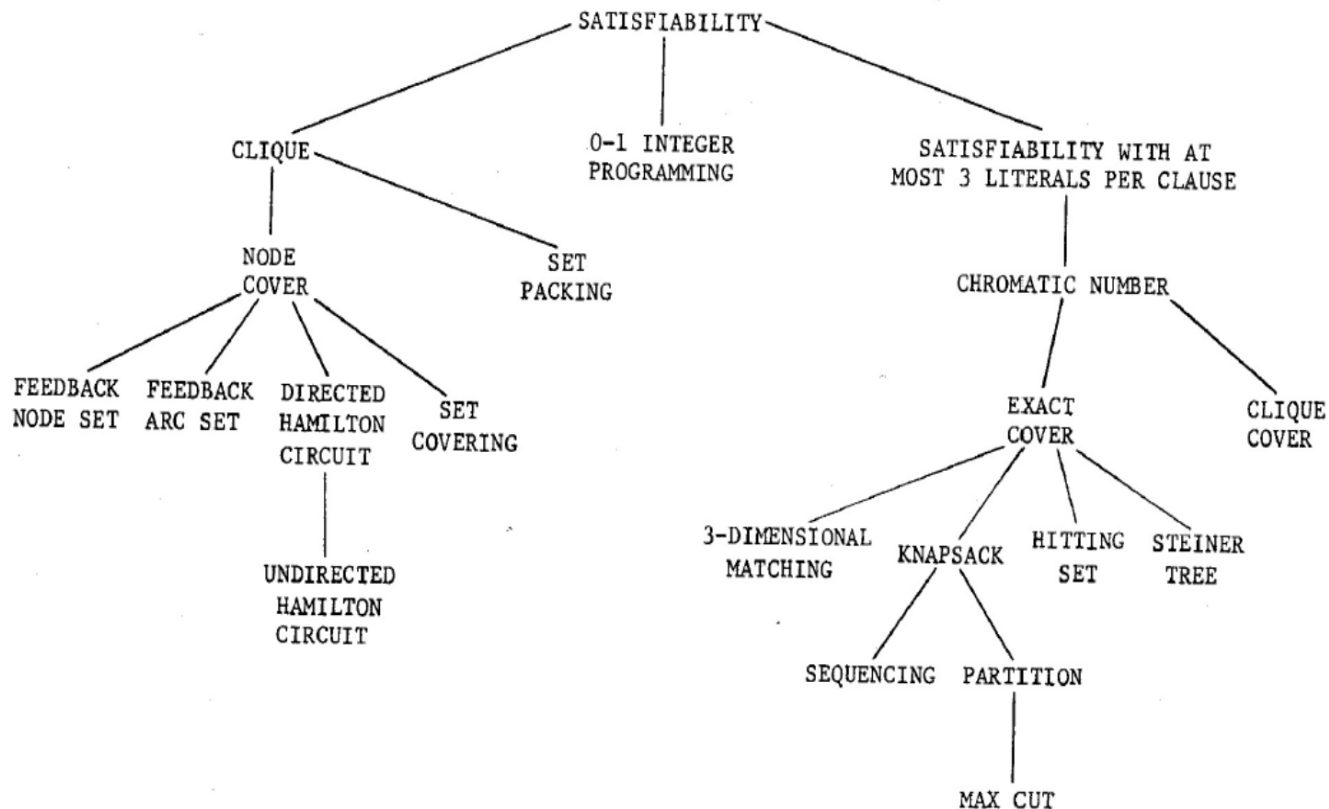


图8.1

21个卡普的NPC问题

- 1972年，理查德·卡普进一步推进，证明了21个不同的NPC问题。



8.6 NP完全问题的求解

当必须求解**NP**完全问题时，有以下几种处理方法：

- 只运行小的问题实例

不是所有问题都接受该方法。如**TSP**问题。

- 解决这个问题的一个不太困难的实例

如，顶点覆盖问题。

若假设图为二部图(图分为两部分，每个子集内部的两个顶点之间无边)，则可在多项式时间内求解。

NP完全问题的近似解法

- 使用动态规划、回溯法或分支限界法求解
- 使用概率算法求解
- 使用启发式算法求解
遗传算法、模拟退火

下面关于 、下面关于 **NP** 问题说法正确的是（ ）

- ☐ **A** **NP** 问题都是不可能解决的
- ☐ **B** **NP**完全问题是**P**类问题的子集
- ☒ **C** **P**类问题包含在**NP**类问题中
- ☐ **D** **NP**类问题包含在**P**类问题中

提交

算法设计与分析

Design & Analysis of Algorithms

第9章 课程总结

算法课程的知识框架

分治策略

动态规划

贪心法

回溯法

分支限界

随机化

使用条件
主要步骤
分析方法
改进途径
典型例子

算法的数学基础：
序列求和、递推方程求解

算法的基本概念： 算法的伪码表示
算法的两种时间复杂度 复杂度函数阶的表示

函数的阶

- 阶的符号: $O, \Omega, \Theta, o, \omega$

- 阶的高低

至少指数级: $2^n, 3^n, n!, \dots$

多项式级: $n, n^2, n \log n, n^{1/2}, \dots$ $\log n$ 的多项式级:
 $\log n, \log^2 n, \dots$

- 注意

阶反映的是大的 n ($n > n_0$) 的情况 可以忽略前面的有限项

序列求和

- 基本求和公式 等比数列
等差数列 调和数级数
- 估计和式的阶
放大，然后估计上界 用积分估计上下界

递推方程求解

- 主要的求解方法

迭代 + 进行序列求和 递归树 + 求和

主定理：注意条件验证

- 一些常见的递推方程的解

$$f(n) = af\left(\frac{n}{b}\right) + d(n)$$

算法设计技术

- **设计技术** 分治策略， 动态规划， 贪心，
回溯， 分支限界， 随机化

- **关注问题** 使用条件

主要设计步骤

时间复杂度分析方法 改进途径

典型例子

分治策略

- **适用条件**: 归约为独立求解子问题
- **设计步骤**: 归约方法, 初始子问题的计算, 子问题解的综合方法. 注意子问题划分均衡, 类型相同。
- **递归算法分析**: 求解递推方程
- **改进途径**: 减少子问题数, 预处理
- **典型问题**: 二分搜索技术, 大整数乘法, 棋盘覆盖, 快速排序, 线性时间选择

动态规划

- **适用条件：** 优化问题, 多步判断求解, 满足优化原则, 子问题重叠
- **设计步骤：** 确定子问题边界, 列关于 目标函数的递推方程及初值；自底向上, 备忘录存储；标记函数及解的 追踪方法
- **复杂度分析：** 备忘录, 递推方程
- **典型问题：** 矩阵链相乘, 背包问题, 序列问题 (最长公共子序列, 编辑距离, 最大子段和)

贪心法

- **适用条件：** 组合优化问题, 多步判断求解, 有贪心选择性质
- **设计步骤：** 局部优化策略的确定及算法正确性证明 (直接证明, 数学归纳法, 交换论证)
- **复杂度分析**
- **典型问题：** 部分背包, 活动安排, 最大子段和, 装载问题.

回溯法

- **适用条件**：搜索或优化问题, 多步判断求解。
- **设计步骤**：确定解向量, 搜索树结构, 深度优先搜索顺序, 结点分支搜索的约束条件与代价函数, 路径存储
- **搜索树结点数估计**
- **复杂度分析**
- **典型问题**: 0/1背包问题, 装载问题, 子集和问题, n 皇后问题, 图的 m 着色问题, 旅行商问题

分支限界法

- **适用条件**: 搜索或优化问题, 多步判断求解
- **设计步骤**: 确定解向量, 搜索树结构, 广度优先搜索顺序, 结点分支搜索的约束条件与代价函数, 路径存储
- **搜索树结点数估计**
- **复杂度分析**
- **典型问题**: 装载问题, 最大团问题, 0/1背包问题, 旅行商问题.

随机化算法

- **适用条件**：随机性选择常比最优选择省时，因此概率算法可在很大程度上降低算法复杂性。。
- **设计步骤**：概率算法允许执行过程中随机选择下一计算步骤
- **概率搜索策略**
- **复杂度分析**
- **典型问题**：舍伍德算法,拉斯维加斯算法，蒙特卡罗算法. 解决快速排序、n皇后问题、主元素问题等

算法设计

- 设计思想：尽量选复杂度低的算法
- 算法实现依赖于数据结构，选择合适的数据结构
- 实际问题中的综合考虑：时空权衡, 实现成本的权衡, ...

下列算法中通常自底向上的方式求解最优是（ ）。

- ☐ A 备忘录法
- ☒ B 动态规划法
- ☐ C 贪心法
- ☐ D 回溯法

提交